

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL X
IMPLEMENTASI JAVA COLLECTIONS



Disusun Oleh:

Ni Putu Haruka Ikeda 105222039

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Pengelolaan data barang merupakan bagian penting dalam sistem pergudangan. Data seperti identitas barang, kategori, dan stok harus disimpan serta dikelola dengan baik agar proses operasional berjalan lancar. Permasalahan yang sering terjadi adalah kesulitan pencarian data, duplikasi kategori, dan kurangnya pencatatan aktivitas stok.

Untuk mengatasi hal tersebut, digunakan Java Collections Framework yang menyediakan struktur data seperti Map, Set, dan List. Pada praktikum ini dibuat program Sistem Manajemen Gudang dengan menerapkan konsep Object Oriented Programming (OOP) dan Collections Framework. Map digunakan untuk menyimpan data barang, Set untuk menyimpan kategori unik, dan List untuk mencatat riwayat aktivitas. Dengan demikian, pengelolaan data menjadi lebih terstruktur dan efisien.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	<i>idBarang</i>	String	Menyimpan kode atau identitas unik barang
2	<i>namaBarang</i>	String	Menyimpan nama barang
3	<i>kategori</i>	String	Menyimpan kategori barang
4	<i>stok</i>	Int	Menyimpan jumlah stok barang yang tersedia
5	<i>databaseBarang</i>	Map<String, Barang>	Menyimpan seluruh data barang dengan ID sebagai key
6	<i>kategoriUnik</i>	Set<String>	Menyimpan daftar kategori barang tanpa duplikasi
7	<i>riwayat</i>	List<String>	Menyimpan riwayat aktivitas atau transaksi dalam sistem

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	<i>Barang()</i>	<i>Constructor</i>	Menginisialisasi data barang berupa ID, nama, kategori, dan stok saat objek dibuat
2	<i>getIdBarang()</i>	Functional (Getter)	Mengambil ID barang

3	getNamaBarang()	Functional (Getter)	Mengambil nama barang
4	getKategori()	Functional (Getter)	Mengambil kategori barang
5	getStok()	Functional (Getter)	Mengambil jumlah stok barang
6	setStok()	Procedural (Setter)	Mengubah jumlah stok barang
7	tambahBarangBaru()	Procedural	Menambahkan barang baru ke sistem
8	tambahStok()	Procedural	Mengubah harga per malam
9	kurangiStok()	Procedural	Mengurangi stok barang jika mencukupi
10	cetakLaporan()	Procedural	Menampilkan kategori, data barang, dan riwayat transaksi
11	hitungTotalBayar()	Functional	Menghitung total biaya berdasarkan jumlah malam menginap
12	tampilkanInfo()	Procedural	Menampilkan informasi lengkap mengenai kamar hotel

IV. Dokumentasi dan Pembahasan Code

4.1 Deklarasi Class dan Atribut

```
public class Barang {
    private String idBarang;
    private String namaBarang;
    private String kategori;
    private int stok;
```

Class Barang digunakan sebagai model untuk menyimpan informasi setiap barang yang terdapat dalam sistem gudang. Atribut idBarang berfungsi sebagai identitas unik barang, namaBarang menyimpan nama barang, kategori menyimpan jenis atau kelompok barang, sedangkan stok digunakan untuk menyimpan jumlah barang yang tersedia. Seluruh atribut dibuat bersifat private untuk menjaga keamanan data dan menerapkan prinsip encapsulation.

4.2 Constructor Barang

```
public Barang(String idBarang, String namaBarang,
String kategori, int stok) {
    this.idBarang = idBarang;
    this.namaBarang = namaBarang;
    this.kategori = kategori;
    this.stok = stok;
}
```

Constructor digunakan untuk menginisialisasi objek Barang ketika pertama kali dibuat. Nilai yang diberikan melalui parameter akan disimpan ke dalam atribut objek menggunakan kata kunci `this`. Dengan demikian, setiap objek barang yang dibuat akan langsung memiliki informasi ID, nama, kategori, dan stok sesuai data yang dimasukkan.

4.3 Getter dan Setter

```
public String getIdBarang() {  
    return idBarang;  
}  
  
public String getNamaBarang() {  
    return namaBarang;  
}  
  
public String getKategori() {  
    return kategori;  
}  
  
public int getStok() {  
    return stok;  
}  
  
public void setStok(int stok) {  
    this.stok = stok;  
}
```

Method getter digunakan untuk mengambil nilai atribut yang bersifat private dari luar class. Sedangkan method setter digunakan untuk memperbarui nilai atribut tertentu, yaitu stok barang. Dengan adanya getter dan setter, akses terhadap data menjadi lebih terkontrol dan sesuai dengan prinsip Object-Oriented Programming.

4.4 Deklarasi Collection pada Class SistemGudang

```
private Map<String, Barang> databaseBarang;  
private Set<String> kategoriUnik;  
private List<String> riwayat;
```

SistemGudang memanfaatkan Java Collections Framework untuk mengelola data secara efisien. Map digunakan sebagai database utama yang menyimpan objek barang dengan ID barang sebagai key sehingga proses pencarian dapat dilakukan dengan cepat. Set digunakan untuk menyimpan kategori barang tanpa duplikasi, sedangkan List digunakan untuk mencatat seluruh aktivitas atau transaksi yang terjadi dalam sistem gudang.

4.5 Method tambahBarangBaru()

```
public void tambahBarangBaru(String id, String nama, String
kategori, int stok) {
    Barang barang = new Barang(id, nama, kategori, stok);
    databaseBarang.put(id, barang);
    kategoriUnik.add(kategori);
    riwayat.add("Barang Baru: " + id + " - " + nama +
        "(" + kategori + ") stok awal " + stok);
}
```

Method ini digunakan untuk menambahkan barang baru ke dalam sistem. Data barang akan dibuat menjadi objek Barang, kemudian disimpan ke dalam Map. Selain itu, kategori barang akan dimasukkan ke dalam Set dan aktivitas penambahan barang dicatat ke dalam List riwayat. Dengan cara ini, sistem dapat menyimpan data barang sekaligus mendokumentasikan aktivitas yang terjadi.

4.6 Method tambahStok()

```
public void tambahStok(String id, int jumlah) {
    if (databaseBarang.containsKey(id)) {
        Barang barang = databaseBarang.get(id);
        barang.setStok(barang.getStok() + jumlah);
        riwayat.add("Barang Masuk: " + id +
            " ditambah " + jumlah + " unit");
        System.out.println("Stok berhasil ditambah.");
    } else {
        System.out.println("ID barang tidak ditemukan!");
    }
}
```

Method ini berfungsi untuk menambahkan jumlah stok suatu barang berdasarkan ID yang diberikan. Sistem terlebih dahulu memeriksa apakah ID barang tersedia dalam database. Jika ditemukan, stok barang akan ditambah sesuai jumlah yang dimasukkan dan aktivitas tersebut dicatat pada riwayat transaksi.

4.7 Method kurangiStok()

```
public void kurangiStok(String id, int jumlah) {  
    if (!databaseBarang.containsKey(id)) {  
        System.out.println("ID barang tidak  
ditemukan!");  
        return;  
    }  
  
    Barang barang = databaseBarang.get(id);  
  
    if (barang.getStok() >= jumlah) {  
        barang.setStok(barang.getStok() - jumlah);  
  
        riwayat.add("Barang Keluar: " + id + "  
" " dikurangi " + jumlah + "  
unit");  
  
        System.out.println("Stok berhasil  
dikurangi.");  
    } else {  
        riwayat.add("Gagal Mengurangi Stok: " + id + "  
" (stok tidak mencukupi)");  
  
        System.out.println("Gagal! Stok tidak  
mencukupi.");  
    }  
}
```

Method ini digunakan untuk mengurangi stok barang. Sebelum pengurangan dilakukan, sistem akan memeriksa keberadaan ID barang dan memastikan bahwa stok yang tersedia mencukupi. Jika syarat terpenuhi, stok akan dikurangi dan transaksi dicatat dalam riwayat. Namun jika stok tidak mencukupi, proses akan ditolak untuk mencegah terjadinya stok negatif.

4.8 Method cetakLaporan()

```
public void cetakLaporan() {  
    System.out.println("\n===== LAPORAN GUDANG =====");  
  
    System.out.println("\nDaftar kategori:");  
    for (String kategori : kategoriUnik) {  
        System.out.println("- " + kategori);  
    }  
  
    System.out.println("\nData Barang:");  
    for (Barang barang : databaseBarang.values()) {  
        System.out.println(  
            barang.getIdBarang() + " | " +  
            barang.getNamaBarang() + " | " +  
            barang.getKategori() + " | Stok: " +  
            barang.getStok()  
        );  
    }  
}
```

```

        System.out.println("\nRiwayat Transaksi:");
        for (String log : riwayat) {
            System.out.println(log);
        }
    }
}

```

Untuk menyajikan data secara komprehensif, terdapat fungsi yang menampilkan seluruh informasi kamar dengan memanfaatkan operator ternary untuk efisiensi logika. Sistem melakukan pengecekan instan terhadap status boolean kamar: jika bernilai true, maka informasi yang muncul adalah "Tersedia", namun jika false, secara otomatis akan tampil sebagai "Terisi".

4.9 Class Main

```

public class Main {
    public static void main(String[] args) {
        SistemGudang gudang = new SistemGudang();

        // Daftarkan minimal 3 barang
        gudang.tambahBarangBaru("B01", "Laptop",
            "Elektronik", 10);
        gudang.tambahBarangBaru("B02", "Mouse",
            "Elektronik", 20);
        gudang.tambahBarangBaru("B03", "Meja",
            "Furniture", 5);

        // Tambah stok berhasil
        gudang.tambahStok("B01", 5);

        // Kurangi stok berhasil
        gudang.kurangiStok("B02", 10);

        // Kurangi stok gagal
        gudang.kurangiStok("B03", 20);

        // Cetak laporan
        gudang.cetakLaporan();
    }
}

```

Class Main berfungsi sebagai program utama yang menjalankan simulasi Sistem Manajemen Gudang. Pada bagian ini dilakukan pendaftaran beberapa barang, penambahan stok, pengurangan stok yang berhasil maupun gagal, serta menampilkan laporan akhir. Simulasi tersebut menunjukkan bagaimana Map, Set, dan List bekerja bersama dalam mengelola data gudang secara terstruktur.

V. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa Java Collections Framework memudahkan pengelolaan data dalam suatu program. Dengan menggunakan Map, data barang dapat disimpan dan dicari dengan cepat berdasarkan ID. Set digunakan untuk menyimpan kategori barang tanpa adanya data yang duplikat, sedangkan List digunakan untuk mencatat riwayat aktivitas yang terjadi dalam sistem.

Selain itu, penerapan konsep Object-Oriented Programming (OOP) melalui class Barang dan SistemGudang membuat program menjadi lebih terstruktur dan mudah dikembangkan. Dengan kombinasi OOP dan Collections Framework, sistem manajemen gudang dapat mengelola data barang, stok, kategori, serta riwayat transaksi secara lebih efisien dan terorganisir.

VI. Daftar Pustaka

Maiga, J., & Emanuel, A. W. R. (2019). Gamification for Teaching and Learning Java Programming for Beginner Students-A Review. *J. Comput.*, 14(9), 590-595.

Wiener, R., & Pinson, L. J. (2000). *Fundamentals of OOP and data structures in Java*. Cambridge University Press.

Poo, D., Kiong, D., & Ashok, S. (2007). *Object-oriented programming and Java*. Springer Science & Business Media.